Lab sheet 26 **Submission Due: 02.12.2024**

Title: WebSockets and Real-time Applications

Aims:

- Introduction to WebSockets
- Building a real-time chat application

Tasks:

- Introduction to WebSockets
- Setting up the server
- Setting up the client
- Start the server and react application

Activities:

1. Introduction to WebSockets

    - **Bidirectional Communication:** Allows both client and server to send messages simultaneously.
    - **Persistent Connection**: Maintains a single, long-lived connection.
    - **Real-time Data Exchange:** Enables real-time updates and notifications.
    - **Low Latency:** Provides efficient and low-latency communication.

2. Setting up the server

    npm install socket.io

```
import { Server } from 'socket.io';

const io = new Server(3000);

io.on('connection', (socket) => {
    console.log('A user connected');

    socket.on('message', (msg) => {
        io.emit('message', msg);
    });

    socket.on('disconnect', () => {
        console.log('A user disconnected'); 1
    });
});
```

3. Setting up the client

npm install socket.io-client

```
import { useState, useEffect } from "react";
import io from "socket.io-client";

const Chat = () => {
  const [messages, setMessages] = useState([]);
  const [message, setMessage] = useState("");
  const [socket, setSocket] = useState(null);

  useEffect(() => {
    const newSocket = io("http://localhost:3000");
    setSocket(newSocket);

    newSocket.on("connect", () => {
      console.log("Connected to server");
    });

    newSocket.on("message", (msg) => {
      console.log("Received message:", msg);
      setMessages([...messages, msg]);
    });

    newSocket.on("disconnect", () => {
      console.log("Disconnected from server");
    });

    return () => {
      newSocket.disconnect();
    };
  }, []);

  const handleSubmit = (e) => {
    e.preventDefault();
    if (socket) {
      socket.emit("message", { text: message, isOwnMessage: true });
      setMessage("");
    } else {
      console.error("Socket not connected");
    }
  };

  return (
    <div>
      <ul>
        {messages.map((msg, index) => (
```

```
          <li
            key={index}
            className={msg.isOwnMessage ? "sent-message" : "received-
message"}
          >
            {msg.text}
          </li>
        ))}
      </ul>
      <form onSubmit={handleSubmit}>
        <input
          type="text"
          value={message}
          onChange={(e) => setMessage(e.target.value)}
        />
        <button type="submit">Send</button>
      </form>
    </div>
  );
};

export default Chat;
```

4. Start the server

   node server.js

5. Start the react app

   npm run dev

Discussion :

- **HTTP vs WebSocket**